# Week 45: Recurrent Neural Networks

A recap of convolutional networks, then sequences and the hidden state, backpropagation through time, why long sequences are hard – and how to do it in TensorFlow/Keras and PyTorch

Morten Hjorth-Jensen

Department of Physics and Center for Computing in Science Education,
University of Oslo, Norway

# Plan for week 45

**Monday lecture (2×45 min)**:

1. **Recap of convolutional networks** (15 min): what week 44 established, and the Keras and PyTorch programs for MNIST.
2. **From feed-forward to recurrent networks**: what we give up, what a sequence model is asked to do, the hidden state.
3. **The architecture**: the simple RNN, folded and unrolled; the RNN as a dynamical system; where the recurrence comes from.
4. **The layers in Keras and PyTorch**: what `SimpleRNN` and `nn.RNN` compute, line by line.
5. **Backpropagation through time**: the recursion, and what `loss.backward()` does with it.
6. **Why long sequences are hard**: vanishing and exploding gradients, measured with autograd; clipping; a network that learns the rule of a damped oscillator.
7. **Three applications**: a sine wave, MNIST row by row, and the adding problem – where the simple RNN runs out of memory.

**Next Monday, first hour**: gated cells (LSTM, GRU) and sound.
**Tuesday**: work on Project 2 (deadline **Monday November 9**); no weekly exercises this week.
**Reading**: Chapter 11, Sections 11.1–11.6 and 11.8; Chapter 10 for the recap. The code is in `week45.ipynb`.

# Recap of week 44, 1: what a CNN is

Convolutional networks are ordinary neural networks with one assumption built into the architecture: *the input is an image*.

- Neurons with learnable weights and biases, dot products and non-linearities; one differentiable function from pixels to class scores; the softmax cross entropy at the end; trained by backpropagation and Adam. Nothing new there.
- **What is new:** each neuron sees only a small region of the input – its receptive field – and the *same* weights (a filter) are applied at every position.

### Two consequences

Far fewer parameters than a dense layer, independent of the image size; and a pattern learned once is recognised wherever it appears (shift equivariance, Theorem 10.7).

# Recap of week 44, 2: the convolution stage

For an input volume $W_1 \times H_1 \times D_1$ and $K$ filters of size $F \times F$ with stride $S$ and padding $P$, each output pixel is a dot product of a filter with a window of the input (cross-correlation, Chapter 10):

$$z_{i,j,k} = b_k + \sum_{c=1}^{D_1} \sum_{u=1}^{F} \sum_{v=1}^{F} w_{u,v,c,k}\ x_{Si+u,\, Sj+v,\, c}.$$

The output volume

$$W_2 = \frac{W_1 - F + 2P}{S} + 1, \qquad D_2 = K$$

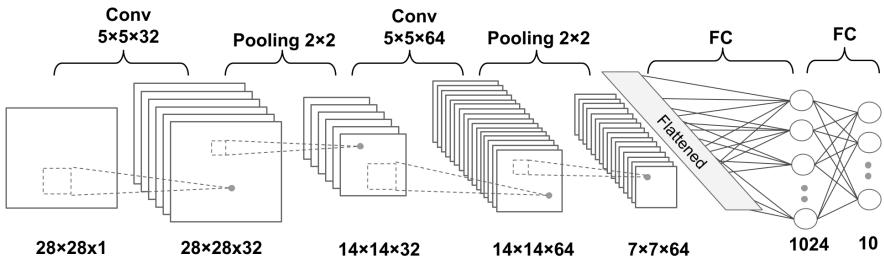
(and the same for $H_2$).

Parameters

$(F \times F \times D_1) \times K + K$: for MNIST with 32 filters of $3 \times 3$, 320 – whatever the image size.

Pooling (a fixed $2 \times 2$ maximum or average) halves width and height between convolution stages; ReLU follows each convolution.

# Recap of week 44, 3: the network we ran (Raschka et al.)




Convolution – pooling – convolution – pooling – flatten – dense – dense:
$28 \times 28 \times 1 \to 28 \times 28 \times 32 \to 14 \times 14 \times 32 \to 14 \times 14 \times 64 \to 7 \times 7 \times 64 \to 1024 \to 10$, with $3 \times 3$ filters. Keras wants the data as $(N, 28, 28, 1)$, channels *last*; PyTorch as $(N, 1, 28, 28)$, channels *first*.

# Recap of week 44, 4: the Keras program

```
model = keras.Sequential([
    keras.Input(shape=(28, 28, 1)),
    layers.Conv2D(32, 3, padding="same", activation="relu"), # 28x28x32, 320 parameters
    layers.MaxPooling2D((2, 2)), # 14x14x32
    layers.Conv2D(64, 3, padding="same", activation="relu"), # 14x14x64, 18496
    layers.MaxPooling2D((2, 2)), # 7x7x64
    layers.Flatten(), # 3136
    layers.Dense(1024, activation="relu"), # 3212288
    layers.Dropout(0.5),
    layers.Dense(10)]) # 10250, logits
model.compile(optimizer=keras.optimizers.Adam(1e-3),
              loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=["accuracy"])
model.fit(Xtr, ytr, epochs=5, batch_size=64)
```

- `Conv2D(K, F, padding="same")`: $K$ filters of $F \times F$, padded so that the size is kept; `MaxPooling2D((2, 2))` halves it.
- `from_logits=True`: the network ends with scores and the loss applies the softmax itself. `Dropout(0.5)` regularises the layer that holds 99% of the parameters.
- The pattern `Sequential` – `compile` – `fit` is the one we will use for recurrent layers today.

# Recap of week 44, 5: the same network in PyTorch

```
net = nn.Sequential(
    nn.Conv2d(1, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2, 2), # (N,32,14,14)
    nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2, 2), # (N,64,7,7)
    nn.Flatten(), nn.Linear(64 * 7 * 7, 1024), nn.ReLU(),
    nn.Dropout(0.5), nn.Linear(1024, 10)) # logits
opt, lossf = torch.optim.Adam(net.parameters(), lr=1e-3), nn.CrossEntropyLoss()
for epoch in range(5):
    net.train() # dropout on
    for idx in torch.randperm(len(Xtr)).split(64):
        opt.zero_grad()
        loss = lossf(net(Xtr[idx]), Ytr[idx])
        loss.backward() # backpropagation, by autograd
        opt.step()
    net.eval() # dropout off for the test accuracy
```

- `nn.Conv2d(C_in, K, F, padding=P)`: input channels, filters, filter size, padding.
- PyTorch shows the loop that `fit` hides: minibatch, loss, `backward()`, a step of Adam. The same loop, unchanged, trains every recurrent network today.
- `nn.CrossEntropyLoss` expects logits; `train()` / `eval()` switch dropout on and off.

# Recap of week 44, 6: what we found on MNIST

| code | model | parameters | test accuracy |
|---|---|---|---|
| PyTorch | dense 784–100–10 (Project 2) | 79 510 | 0.9720 |
| Keras | dense 784–100–10 | 79 510 | 0.9720 |
| PyTorch | CNN of the listings | 3 241 354 | 0.9906 |
| Keras | CNN of the listings | 3 241 354 | 0.9909 |
| PyTorch | CNN with global average pooling | 19 466 | 0.9336 |
| Keras | CNN with global average pooling | 19 466 | 0.9398 |

Five epochs, Adam $10^{-3}$, batches of 64, seed 1; full curves in `week44_mnist.pdf`.

- The CNN makes 94 errors on the 10 000 test images where the Project 2 dense network makes 280; the two libraries agree to the noise of a stochastic optimiser.
- On the scikit-learn digits placed at a random position the dense network lost 26 points and the CNN 9: convolution pays when patterns can appear *anywhere on a grid*.

# From space to time

Week 44

A filter does not care how large the image is, and finds a pattern wherever it is: *parameter sharing along the spatial axes*.

Today

A recurrent cell does not care how long the sequence is, and applies the same update at every step: *parameter sharing along the time axis*.

Same idea, same consequences – a parameter count independent of the input size, and equivariance (to shifts in space, to shifts in time) – and, as we will see, a new difficulty that convolution does not have: the sharing is *recursive*, each step feeding the next.

# What we have assumed so far, 1: independent samples

Almost everything since week 35 – the bias-variance decomposition, the bootstrap, cross-validation, the statistical toolbox of Chapter 2 – leans on the training examples being *independent and identically distributed*.

For a sequence this is simply false. The value of a detector trace at one sample is strongly informative about the next; that is the entire content of the data.

### The consequence

A model that treats the samples as independent has thrown the signal away before it starts.

# What we have assumed so far, 2: a fixed input size

A feed-forward network has a fixed input dimension. A convolutional network (week 44) has one too, up to the pooling that precedes its dense layers.

Sequences come in different lengths – a measurement that ran for ten seconds and one that ran for an hour, a spoken word of 0.3 s and one of 0.6 s – and we would like *one* model to handle all of them.

# The move of this chapter

Week 44

We restricted the dense layer so that it respects the *spatial* structure of an image, and obtained *convolution*.

Today

We make the same kind of restriction for data with a *temporal* structure, and obtain *recurrence*: process one element at a time and carry a state forward, so that the same parameters serve a sequence of any length.

# Sequences in the physical sciences

Ordered, correlated data are the norm rather than the exception:

- detector read-out streams and waveform traces;
- climate and seismic records;
- molecular-dynamics trajectories and spectroscopy scans;
- state estimation in control and feedback;
- the solutions of differential equations, which are time series by construction;
- sound and speech (next Monday).

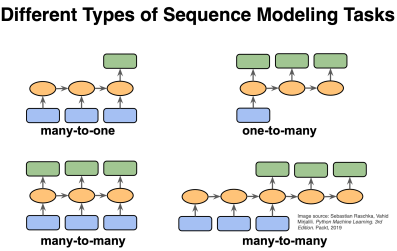
Sequential need not mean temporal (text, genomes), but our examples are time series. We write $x_{1:T} = (x_1, \ldots, x_T)$.

# Three tasks, Eq. (11.1)

$$p(x_{1:T}), \qquad p(y_{1:T} \mid x_{1:T}), \qquad p(y \mid x_{1:T}).$$

- modelling the sequence itself;
- mapping a sequence to a sequence: filtering, forecasting (*many-to-many*);
- mapping a sequence to one label: classification, regime detection (*many-to-one*).

Today: the oscillator is many-to-many; the sine wave, MNIST and the adding problem are many-to-one.




Figures in this part from Raschka and Mirjalili, *Python Machine Learning*.

# What is a recurrent neural network?

In a feed-forward network the information flows in one direction only, from the input to the output.

A recurrent network also has connections pointing *backward*: the hidden layer is connected to *itself*. The hidden state at step $t$ receives two inputs – the data $x^{(t)}$ and its own value $h^{(t-1)}$ from the previous step.

## What this buys

A *memory* across time steps, and the same layer for the first element of a sequence and the thousandth.

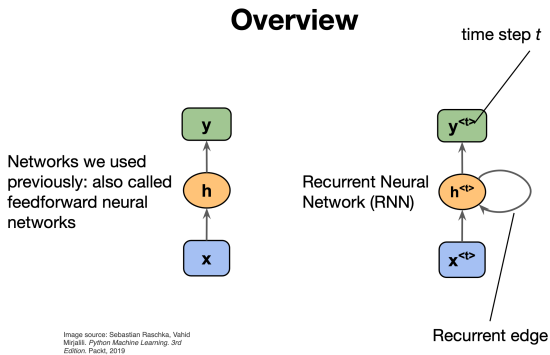



Image source: Sebastian Raschka, Vahid Mirjalili. *Python Machine Learning. 3rd Edition.* Packt, 2019

# Fold and unfold: two drawings of the same computation

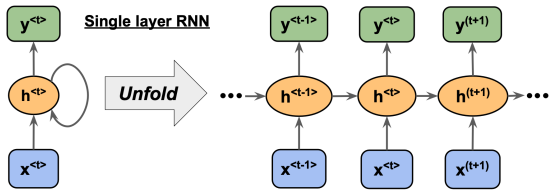



Image source: Sebastian Raschka, Vahid Mirjalili. *Python Machine Learning. 3rd Edition*. Packt, 2019

## Folded (left)

A single layer with an arrow leaving it and returning to itself. This is how the network is *stored*: one copy of the weights.

## Unrolled (right)

The same layer drawn once per time step. This is how the network is *executed* – and, as we will see, how it is *differentiated*.

# The hidden state: a latent state-space model

The chain rule of probability factorises the first task exactly, Eq. (11.2),

$$p(x_{1:T}) = \prod_{t=1}^{T} p(\mathrm{x}_t \mid x_{1:t-1}),$$

exact and useless as it stands: the conditioning set grows without bound. What makes it tractable is the assumption that everything relevant in $x_{1:t-1}$ can be summarised in a finite-dimensional *state* $\mathrm{h}_t$, updated recursively, Eq. (11.3):

$$\boxed{\mathrm{h}_t = f_\theta(\mathrm{h}_{t-1}, \mathrm{x}_t), \qquad \hat{\mathrm{y}}_t = g_\theta(\mathrm{h}_t).}$$

## This should look familiar

Equation (11.3) is the structure of a *Markovian dynamical system*. The Kalman filter is the case where $f_\theta$ is linear and the noise Gaussian. A recurrent neural network is the case where $f_\theta$ and $g_\theta$ are neural networks – the update map is *learned*.

# The simple RNN, Eq. (11.4)

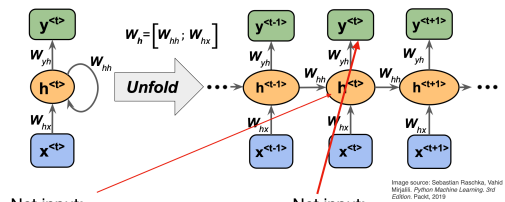
One affine map and one activation per step:

$$\mathrm{a}^{(t)} = \mathrm{U}\mathrm{x}^{(t)} + \mathrm{W}\mathrm{h}^{(t-1)} + \mathrm{b},$$
$$\mathrm{h}^{(t)} = \sigma_h\left(\mathrm{a}^{(t)}\right),$$
$$\mathrm{o}^{(t)} = \mathrm{V}\mathrm{h}^{(t)} + \mathrm{c},$$
$$\hat{\mathrm{y}}^{(t)} = \sigma_y\left(\mathrm{o}^{(t)}\right),$$

$\mathrm{U} \in \mathbb{R}^{n_h \times n_x}$ maps the input into the state, $\mathrm{W} \in \mathbb{R}^{n_h \times n_h}$ the state to itself, $\mathrm{V} \in \mathbb{R}^{n_y \times n_h}$ reads the state out; $\mathrm{h}^{(0)} = 0$.

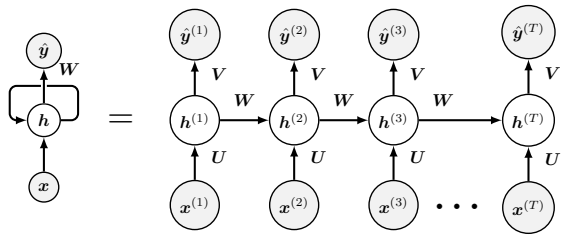
$\sigma_h$ is almost always tanh (Part 6 says why); $\sigma_y$ is the identity for regression, the softmax for classification.

**Weight matrices in a single-hidden layer RNN**




Raschka's notation: $\mathrm{U} = \mathrm{W}_{hx}$, $\mathrm{W} = \mathrm{W}_{hh}$, $\mathrm{V} = \mathrm{W}_{yh}$.

# The same arrays at every step (Figure 11.1)




The parameters do not depend on $t$

Parameter sharing along the time axis, exactly as convolution shares parameters along the spatial axes (week 44): the parameter count is independent of the sequence length, and the model is equivariant to a shift in time.

# Two more properties of Eq. (11.4)

The state is a bottleneck

Everything the network will ever know about $x_{1:t}$ when it predicts $\hat{\mathrm{y}}^{(t+1)}$ must fit in the $n_h$ numbers of $\mathrm{h}^{(t)}$.

The cost accumulates over the sequence, Eq. (11.5)

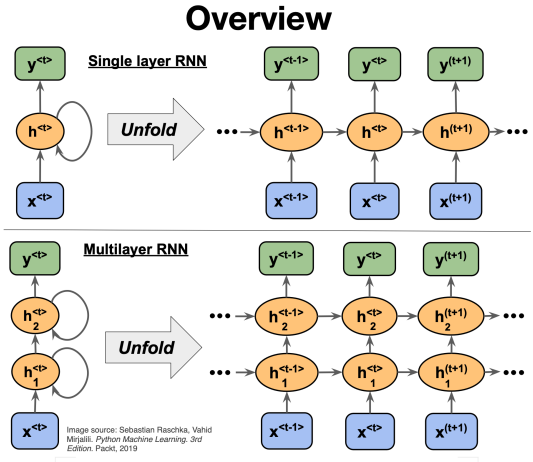
$$\mathcal{L} = \frac{1}{T}\sum_{t=1}^{T} L^{(t)}, \qquad L^{(t)} = \tfrac{1}{2}\|\hat{\mathrm{y}}^{(t)} - \mathrm{y}^{(t)}\|^2$$

for regression, the cross entropy for classification. In the sequence-to-label case only $L^{(T)}$ is non-zero – the hardest case of all, as Part 6 will show.

# Stacking: a multilayer RNN

The hidden state of one recurrent layer can be the input sequence of the next. Each layer has its own $U, W, b$ and is unrolled along the same time axis, so the unrolled graph becomes a grid: depth in layers downwards, depth in time sideways.

In the libraries: `num_layers=2` in `nn.RNN`, or two `SimpleRNN` layers with `return_sequences=True` in Keras (Part 4).



# The RNN as a discrete-time dynamical system (Section 11.2.1)

Set the input to zero. Equation (11.4) becomes an autonomous map, Eq. (11.6),

$$\mathsf{h}^{(t)} = F\left(\mathsf{h}^{(t-1)}\right) := \sigma_h\left(\mathsf{W}\mathsf{h}^{(t-1)} + \mathsf{b}\right).$$

Fixed points satisfy $\mathsf{h}^\star = F(\mathsf{h}^\star)$; their stability is decided by the Jacobian, Eq. (11.7),

$$\mathsf{J} = \left.\frac{\partial F}{\partial \mathsf{h}}\right|_{\mathsf{h}^\star} = \operatorname{diag}(\sigma_h'(\mathsf{a}^\star))\,\mathsf{W},$$

attracting if the spectral radius $\rho(\mathsf{J}) < 1$, repelling if $\rho(\mathsf{J}) > 1$. In two dimensions the map can have limit cycles, in three or more it can be chaotic.

# Why this matters, twice

### What a recurrent network can represent

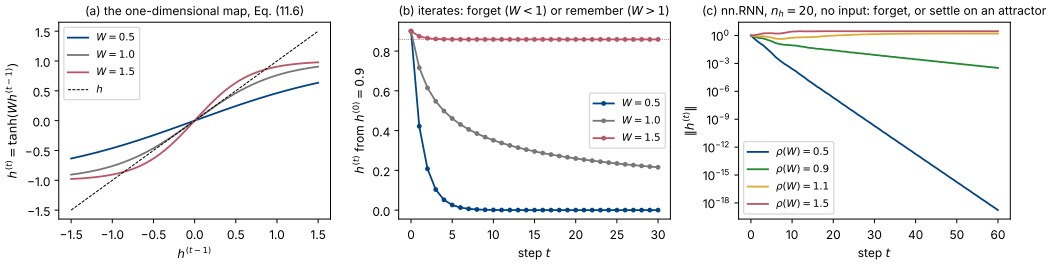
Oscillations, decay to a point attractor, chaotic wandering.

### What it does with information

Strongly contracting dynamics forget the initial condition quickly and cannot carry information far; expanding dynamics are unstable.

Equation (11.7) is also exactly the quantity that will control the gradient in Part 6: the stability of the forward dynamics and the survival of the backward gradient are the *same question*.

# The one-dimensional map, and an untrained `nn.RNN` (`week45_dynamics.pdf`)




(a) $h \mapsto \tanh(Wh)$ against the diagonal: for $W < 1$ the only fixed point is 0, for $W > 1$ two new ones appear. (b) Iterates from $h^{(0)} = 0.9$: at $W = 0.5$ the state is gone after ten steps, at $W = 1.5$ it sits on $h^\star = \pm 0.8586$ for ever. (c) A PyTorch `nn.RNN` with $n_h = 20$, zero input, W scaled to a given spectral radius: at $\rho(\mathrm{W}) = 0.5$ the norm of the state falls by nineteen orders of magnitude in sixty steps; at $\rho > 1$ it settles on an attractor of order one.

# A differential equation (Section 11.3)

The recurrence of Eq. (11.4) can look arbitrary. It is not. Newton's equation for a damped harmonic oscillator, scaled to dimensionless time, Eq. (11.8):

$$\frac{\mathrm{d}^2 x}{\mathrm{d}t^2} + \eta \frac{\mathrm{d}x}{\mathrm{d}t} + x(t) = F(t),$$

with $\eta$ the damping and $F$ an external force. As always we rewrite it as two coupled first-order equations, Eq. (11.9),

$$v = \frac{\mathrm{d}x}{\mathrm{d}t}, \qquad \frac{\mathrm{d}v}{\mathrm{d}t} = F(t) - \eta v(t) - x(t),$$

and integrate from $(x_0, v_0)$ – by hand for $F = 0$, by Runge-Kutta in general.

# Discretise, and the recurrence appears

Forward Euler for the velocity gives step $i+1$ from step $i$, Eq. (11.10):

$$v_{i+1} = v_i + \Delta t\,(F_i - \eta v_i - x_i) =: h_i(x_i, v_i, F_i)\,,$$

a function of the *current state* and the *current input*. But $v_i$ was itself produced by the same rule one step earlier, Eq. (11.11), so substituting, Eq. (11.12), and renaming, Eq. (11.13),

$$v_{i+1} = h_i(x_i,\, h_{i-1},\, F_i) \qquad \Longrightarrow \qquad \mathrm{h}_i = h(\mathrm{x}_i, \mathrm{h}_{i-1})\,,$$

which is Eq. (11.3) exactly.

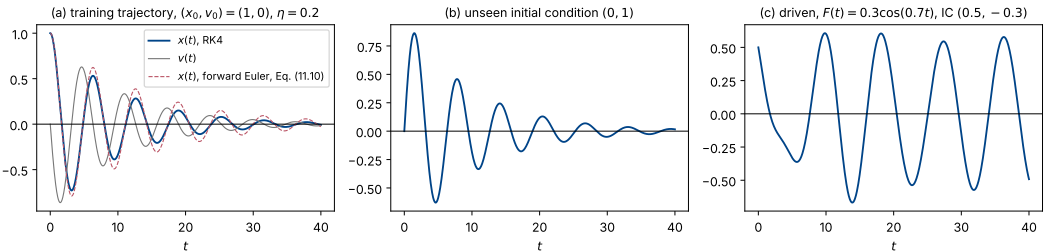
# A recurrent network is a learned, non-linear time-stepper

The correspondence

- the hidden state plays the role of the integrator's internal state;
- W plays the role of $1 - \eta \Delta t$;
- the training data supply what the differential equation would otherwise have to tell us.

The output $\hat{y}_i$ is compared with a target through a cost – the target being a measurement, or the solution of the equation.

# The data for today's experiment (`week45_oscillator_data.pdf`)




Fourth-order Runge-Kutta with $\eta = 0.2$ on $t \in [0, 40]$, 800 points. Forward Euler at the same $\Delta t = 0.05$ (dashed) is visibly off by the second period – an error of 0.1 – a first hint of what a *learned* stepper can improve on. In Part 6 the network sees only the blue curve of (a); (b) and (c) are the tests.

# Compare week 42: two ways to solve a differential equation with a network

**Chapter 9 (week 42)**

The equation was *known* and supplied the cost through its residual; no data were needed; the network learned a function of continuous time.

**Chapter 11 (today)**

The equation is *unknown* and the data supply the cost; the network learns a discrete update rule.

The first is for known physics and absent data, the second for present data and unknown physics.

# Two libraries, one layer

| Equation | TensorFlow/Keras | PyTorch |
|---|---|---|
| simple RNN, Eq. (11.4) | `layers.SimpleRNN(units)` | `nn.RNN(input_size, hidden_size)` |
| gated cells (next Monday) | `layers.LSTM`, `layers.GRU` | `nn.LSTM`, `nn.GRU` |
| read-out $Vh + c$ | `layers.Dense(n_y)` | `nn.Linear(hidden_size, n_y)` |
| clipping, Eq. (11.23) | `Adam(lr, clipnorm=1.0)` | `nn.utils.clip_grad_norm_(params, 1.0)` |
| BPTT | inside `model.fit` | `loss.backward()` |

The one convention to remember: the data layout

Both libraries want a 3-tensor of shape `(samples, timesteps, features)` $= (N, T, n_x)$: $N$ sequences, each $T$ steps of $n_x$ numbers. In PyTorch this needs `batch_first=True` (the default is $(T, N, n_x)$). A single time series is *cut into windows* to make the first axis.

# Keras: a minimal recurrent model

```
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential([
    keras.Input(shape=(None, 1)), # (T, n_x): T may vary, one feature per step
    layers.SimpleRNN(24, return_sequences=True), # Eq. (11.4): h_t for every t, n_h = 24
    layers.Dense(1), # V h_t + c, applied at every step
])
model.compile(optimizer=keras.optimizers.Adam(5e-3, clipnorm=1.0), loss="mse")
model.summary() # 24*(1+24+1) + 25 = 649 parameters
history = model.fit(X_train, Y_train, epochs=300, batch_size=4, verbose=0)
```

`X_train` has shape $(N, T, 1)$ and `Y_train` the shape $(N, T, 1)$ of the targets – here one target per step, the sequence-to-sequence case. Six lines; the next frame reads them one by one.

# Keras: what each line does

`Input(shape=(None, 1))` the shape of *one* sample without the batch axis: $T$ steps (`None` = any length) of $n_x = 1$ features.

`SimpleRNN(24)` one recurrent layer with $n_h = 24$: U ($1 \times 24$, stored transposed as `kernel`), W ($24 \times 24$, `recurrent_kernel`), b (`bias`); tanh by default; $\mathrm{h}^{(0)} = 0$.

`return_sequences=True` output all of $\mathrm{h}^{(1)}, \ldots, \mathrm{h}^{(T)}$, shape $(N, T, 24)$. With `False` (the default) only $\mathrm{h}^{(T)}$, shape $(N, 24)$ – the sequence-to-label case.

`Dense(1)` $\mathrm{V}\mathrm{h} + \mathrm{c}$. Applied to a 3-tensor it acts on the last axis, i.e. at every time step with the same V, c.

`compile(...)` the optimiser (Adam, week 38) and the cost; `clipnorm=1.0` is gradient clipping, Eq. (11.23), on the whole gradient vector.

`fit(...)` forward pass, backpropagation through time and the update, `epochs` times over all windows in mini-batches of `batch_size` windows.

# Keras: the knobs you will actually use

- `SimpleRNN` → `LSTM` → `GRU` (next Monday): change one word; the shapes of input and output are the same.
- `return_sequences=True` on every layer but the last to *stack* recurrent layers; `return_state=True` to also get the final state.
- `Input(shape=(T, n_x))` with a fixed $T$ when all windows have the same length (faster); `None` when they do not.
- Variable-length sequences in one batch: pad, and put a `layers.Masking(mask_value=0.0)` first so that padded steps are skipped.
- Classification from the last state: `Dense(n_classes)` with `SparseCategoricalCrossentropy(from_logits=True)` – the softmax and the cross entropy of week 41 in one object.

# PyTorch: the same model

```
import torch, torch.nn as nn

class Forecaster(nn.Module):
    def __init__(self, n_h=24):
        super().__init__()
        self.rnn = nn.RNN(input_size=1, hidden_size=n_h, batch_first=True) # Eq. (11.4), tanh
        self.fc = nn.Linear(n_h, 1) # V h_t + c

    def forward(self, x): # x: (N, T, 1)
        H, h_T = self.rnn(x) # H: (N, T, n_h) = every h_t; h_T: (1, N, n_h)
        return self.fc(H) # (N, T, 1): one prediction per step

model = Forecaster()
print(sum(p.numel() for p in model.parameters())) # 673 (two bias vectors, see below)
```

PyTorch separates the *definition* of the network (a class with a `forward`) from its *training* (a loop you write). Keras hides the loop in `fit`; PyTorch shows it – which is why we use it when we want to see the gradients.

# PyTorch: what each line does

`nn.RNN(input_size, hidden_size, batch_first=True)` Eq. (11.4) with tanh (`nonlinearity="relu"` is the other option). `batch_first=True` makes the input $(N, T, n_x)$. `num_layers=2` stacks two layers.

`self.rnn(x)` returns a pair `H` of shape $(N, T, n_h)$ with all hidden states, and `h_T` of shape $(\texttt{num_layers}, N, n_h)$ with the last one. You choose: all of `H` for sequence-to-sequence, `H[:, -1, :]` for sequence-to-label.

the parameters `weight_ih_l0` is U ($n_h \times n_x$), `weight_hh_l0` is W, and there are *two* biases `bias_ih_l0` and `bias_hh_l0` whose sum is our b – hence $673 = 649 + 24$.

`nn.Linear(n_h, 1)` $\mathrm{V}\mathsf{h} + \mathrm{c}$ on the last axis, exactly like Keras's `Dense`.

`nn.LSTM`, `nn.GRU` (next Monday) same calls, same shapes.

# PyTorch: the training loop, line by line

```
opt = torch.optim.Adam(model.parameters(), lr=5e-3)
lossfn = nn.MSELoss()
for epoch in range(300):
    for idx in torch.randperm(len(X_train)).split(4): # shuffle windows, batches of 4
        opt.zero_grad() # forget last batch's gradients
        loss = lossfn(model(X_train[idx]), Y_train[idx]) # forward pass, Eq. (11.5)
        loss.backward() # BPTT, Eqs. (11.16)-(11.18)
        gnorm = nn.utils.clip_grad_norm_(model.parameters(), 1.0) # Eq. (11.23); returns ||g||
        opt.step() # Adam update, week 38
```

- `randperm(...).split(4)`: the *windows* are shuffled, the time order inside each window is not.
- `zero_grad()`: PyTorch *accumulates* gradients into `.grad`, so each batch starts by clearing them.
- `loss.backward()`: automatic differentiation through the unrolled graph – Part 5 says what it computes.
- `clip_grad_norm_` rescales the whole gradient in place if its norm exceeds 1 and returns the norm *before* clipping – which is how we will look at it.

# Counting parameters in both libraries

```
layer Keras PyTorch formula n_h(n_x+n_h+1) n_x=1, n_h=24
SimpleRNN / nn.RNN 624 648 624
+ Dense(1) / Linear 25 25 25 (n_y(n_h+1))
whole model 649 673
```

- Keras's `SimpleRNN` prints exactly Eq. (11.4): $n_h(n_x + n_h + 1) = 24 \cdot 26 = 624$. Use `model.summary()` as a check of what you built.
- PyTorch is larger by $n_h$: the second bias vector. The model is the same, only the parameterisation is redundant.
- Neither count contains $T$. A dense network fed the flattened window of $T = 40$ values with 24 hidden nodes has 1009 parameters, and 9649 at $T = 400$ – and must be retrained when $T$ changes.

# Same weights, same outputs: the conventions (Section 11.8.3)

```
rnn = nn.RNN(n_x, n_h, batch_first=True) # PyTorch, random weights
U = rnn.weight_ih_l0.detach().numpy() # (n_h, n_x)
W = rnn.weight_hh_l0.detach().numpy() # (n_h, n_h)
b = (rnn.bias_ih_l0 + rnn.bias_hh_l0).detach().numpy() # one bias in Eq. (11.4)
k = keras.Sequential([keras.Input(shape=(T, n_x)), layers.SimpleRNN(n_h, return_sequences=True)])
k.layers[0].set_weights([U.T, W.T, b]) # Keras stores the transposes
print(np.max(np.abs(rnn(X)[0].detach().numpy() - k(X.numpy()).numpy()))) # 1.2e-07
```

Two conventions differ: PyTorch has two biases where Eq. (11.4) has one, and Keras stores U and W transposed (`kernel` is $(n_x, n_h)$). With those reconciled the hidden states agree to $10^{-7}$ – single precision – for the same inputs: the two libraries compute the same function, and it is Eq. (11.4).

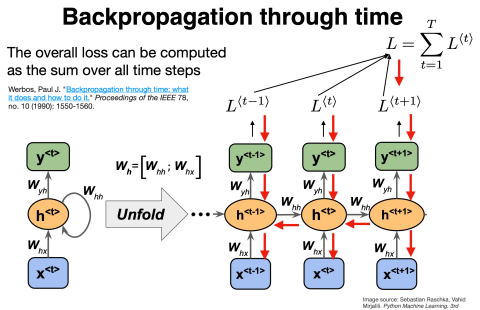
# Training: unroll, and backpropagate (Section 11.4)

## The idea

A recurrent network run for $T$ steps *is* a feed-forward network of depth $T$ in which every layer shares the same weights. Backpropagation (week 41) applies unchanged; the only novelty is that the gradients from every step must be *summed* into the same parameter arrays.

## In the time domain

1. The forward pass builds a *stack* of the activities at each time step.
2. The backward pass peels the stack to compute the error derivatives at each time step.
3. The derivatives at all times are added together for each weight.




The red arrows are the backward pass; the loss is a sum over time (Werbos 1990).

# The output layer and the last hidden state

Differentiating the cost (11.5) gives, for each $t$, Eq. (11.14),

$$(\nabla_{\mathsf{o}^{(t)}}\mathcal{L})_i = \frac{\partial \mathcal{L}}{\partial L^{(t)}} \frac{\partial L^{(t)}}{\partial o_i^{(t)}},$$

which for the quadratic cost and a linear $\sigma_y$ is simply $(\hat{\mathsf{y}}^{(t)} - \mathsf{y}^{(t)})/T$ – the output error of week 41, divided by the number of steps.

At $t = T$ the state influences the cost *only* through the output at that step, Eq. (11.15):

$$\nabla_{\mathsf{h}^{(T)}}\mathcal{L} = \mathsf{V}^{\mathsf{T}} \nabla_{\mathsf{o}^{(T)}}\mathcal{L}.$$

This is where the backward pass starts: at the last step, with nothing arriving from the future.

# Earlier hidden states have two routes to the cost

For $t < T$ the state $\mathsf{h}^{(t)}$ reaches the cost through the output at step $t$ *and* through the state at step $t+1$. The chain rule gives two terms, Eq. (11.16),

$$\nabla_{\mathsf{h}^{(t)}} \mathcal{L} = \left( \frac{\partial \mathsf{h}^{(t+1)}}{\partial \mathsf{h}^{(t)}} \right)^{\mathsf{T}} \nabla_{\mathsf{h}^{(t+1)}} \mathcal{L} + \left( \frac{\partial \mathsf{o}^{(t)}}{\partial \mathsf{h}^{(t)}} \right)^{\mathsf{T}} \nabla_{\mathsf{o}^{(t)}} \mathcal{L}$$

and with Eq. (11.4) the two Jacobians are explicit, Eq. (11.17):

$$\nabla_{\mathsf{h}^{(t)}} \mathcal{L} = \mathsf{W}^{\mathsf{T}} \operatorname{diag}\left( \sigma_h'(\mathsf{a}^{(t+1)}) \right) \nabla_{\mathsf{h}^{(t+1)}} \mathcal{L} + \mathsf{V}^{\mathsf{T}} \nabla_{\mathsf{o}^{(t)}} \mathcal{L}.$$

Equations (11.15) and (11.17), run backwards from $t = T$ to $t = 1$, are the whole of backpropagation through time.

# Compare week 41

**The first term**

is the $\delta$-recursion of Eq. (8.64): $W^T$ times the error of the next layer times the derivative of the activation – with "next layer" meaning "next time step" and the same W at every step.

**The second term**

is new: every layer of the unrolled network has its own output and its own piece of the cost.

# The parameters: every step contributes, Eq. (11.18)

Because the same matrices act at every step, each accumulates a contribution from every step. With $\boldsymbol{\delta}^{(t)} = \operatorname{diag}(\sigma_h'(\mathsf{a}^{(t)}))\,\nabla_{\mathsf{h}^{(t)}}\mathcal{L}$,

$$\nabla_{\mathsf{c}}\mathcal{L} = \sum_t \nabla_{\mathsf{o}^{(t)}}\mathcal{L}, \qquad \nabla_{\mathsf{V}}\mathcal{L} = \sum_t (\nabla_{\mathsf{o}^{(t)}}\mathcal{L})\,\mathsf{h}^{(t)\mathsf{T}},$$

$$\nabla_{\mathsf{b}}\mathcal{L} = \sum_t \boldsymbol{\delta}^{(t)}, \qquad \nabla_{\mathsf{W}}\mathcal{L} = \sum_t \boldsymbol{\delta}^{(t)}\mathsf{h}^{(t-1)\mathsf{T}}, \qquad \nabla_{\mathsf{U}}\mathcal{L} = \sum_t \boldsymbol{\delta}^{(t)}\mathsf{x}^{(t)\mathsf{T}}.$$

Each term is the week-41 formula "error times the input of that layer", Eqs. (8.67)–(8.68); the sum over $t$ is the weight sharing.

# The backward pass is linear

There is an asymmetry between the two passes that is easy to miss.

### Forward

Non-linear: $\sigma_h$ squashes, which keeps the activities bounded and is what prevents them from exploding.

### Backward

*Completely linear* in the incoming gradient: Eq. (11.16) is a matrix acting on $\nabla_{\mathbf{h}^{(t+1)}} \mathcal{L}$, the matrix fixed by the forward pass. Double the error at the final step and every derivative doubles.

Nothing bounds the backward pass, which is precisely why it can explode when the forward pass cannot.

# What `loss.backward()` computes

We do not write Eqs. (11.15)–(11.18) ourselves: both libraries record the unrolled graph during the forward pass and differentiate it backwards – reverse-mode automatic differentiation (week 38). Two consequences:

- the gradient is *exactly* the recursion above, to rounding;
- it is a fixed linear map of the output error, which we can test:

```
rnn, fc = nn.RNN(2, 6, batch_first=True).double(), nn.Linear(6, 1).double()
for scale in (1.0, 2.0): # output error e, then 2e
    for q in params: q.grad = None
    loss = 0.5 * ((fc(rnn(X)[0]) - target(scale)) ** 2).sum()
    loss.backward() # BPTT
    grads.append([q.grad.clone() for q in params])
# max |g(2e) - 2 g(e)| / max|g(2e)| = 4.3e-16 (float64: exact)
```

In Keras the same thing happens inside `model.fit`; to see it, use `tf.GradientTape()` around the forward pass and call `tape.gradient(loss, model.trainable_variables)`.

# What travels backwards in the sequence-to-label case

Suppose the cost is non-zero only at the last step ($L^{(t)} = 0$ for $t < T$: a label for the whole window, `H[:, -1, :]` or `return_sequences=False`). Then the second term of Eq. (11.16), $\mathsf{V}^\mathsf{T} \nabla_{\mathsf{o}^{(t)}} \mathcal{L}$, is zero for every $t < T$, and the gradient that reaches $t = 1$ is

$$\prod_{k=2}^{T} \mathsf{W}^\mathsf{T} \operatorname{diag}\left(\sigma_h'(\mathsf{a}^{(k)})\right)$$

applied to the single output error: $T - 1$ factors of $\mathsf{W}^\mathsf{T}$, each multiplied by a $\tanh'$ that is at most one.

That product is the subject of the next part. `loss.backward()` computes it faithfully – which is the problem, not the cure.

# A product of Jacobians (Section 11.5)

Iterating Eq. (11.17) from $T$ back to $t$ produces a *product* of Jacobians, one factor per step. It is this product, and not anything about the architecture as such, that makes recurrent networks hard to train.

Theorem 11.1 (vanishing and exploding gradients)

Let $\mathrm{h}^{(t)} = \sigma_h(\mathrm{U}\mathrm{x}^{(t)} + \mathrm{W}\mathrm{h}^{(t-1)} + \mathrm{b})$ with $|\sigma_h'(z)| \le \mu$. Then for $t < T$, Eq. (11.19),

$$\frac{\partial \mathrm{h}^{(T)}}{\partial \mathrm{h}^{(t)}} = \prod_{k=t+1}^{T} \operatorname{diag}\!\left(\sigma_h'(\mathrm{a}^{(k)})\right) \mathrm{W},$$

and its spectral norm obeys, Eq. (11.20),

$$\left\| \frac{\partial \mathrm{h}^{(T)}}{\partial \mathrm{h}^{(t)}} \right\|_2 \le \left(\mu\, \sigma_{\max}(\mathrm{W})\right)^{T-t}.$$

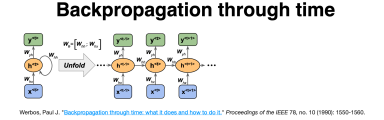
If $\mu\sigma_{\max} < 1$ the contribution of step $t$ decays *exponentially* in the lag $T - t$; if $\mu\sigma_{\max} > 1$ it may grow exponentially.

# The proof in one line, and the picture

One chain-rule factor per step; the spectral norm is submultiplicative; each factor has norm at most

$$\| \operatorname{diag}(\sigma_h') \|_2 \, \|\mathrm{W}\|_2 \leq \mu \, \sigma_{\max}(\mathrm{W}).$$

The bound is *sufficient* for vanishing, not necessary: it uses $\sigma_{\max}(\mathrm{W})$, and for a random (non-normal) W this is larger than $\rho(\mathrm{W})$. We will see the gap in the measurement.



# What the activation function decides

### Why tanh

For tanh, $\mu = 1$, attained only at the origin.

### Why not the logistic function

$\mu = 1/4$: a logistic recurrent network loses a factor of at least four per step *regardless of the weights* – two bits of gradient per step, $10^{35}$ over sixty steps.

### Why not ReLU

$\mu = 1$ too, but the derivative is 0 or 1 rather than a number approaching 1 smoothly, and a ReLU recurrent network is prone to explosion. This is why the rectified family, dominant everywhere else in the course, is largely absent from recurrent architectures.

# Corollary 11.2: the memory horizon

If $\mu\sigma_{\max}(W) < 1$, define Eq. (11.21),

$$\tau = \frac{-1}{\log(\mu\,\sigma_{\max}(W))} > 0,$$

so that the bound reads $e^{-(T-t)/\tau}$: the influence of step $t$ on step $T$ decays with a characteristic scale of $\tau$ steps and falls below $\varepsilon$ once $T - t > \tau \log(1/\varepsilon)$.

## A design parameter

$\tau$ converts the theorem from a qualitative warning into a number: *it is the length of sequence the network can actually use*. We measure it two frames on, and the answer is smaller than one expects.

# Truncated BPTT, Eq. (11.22)

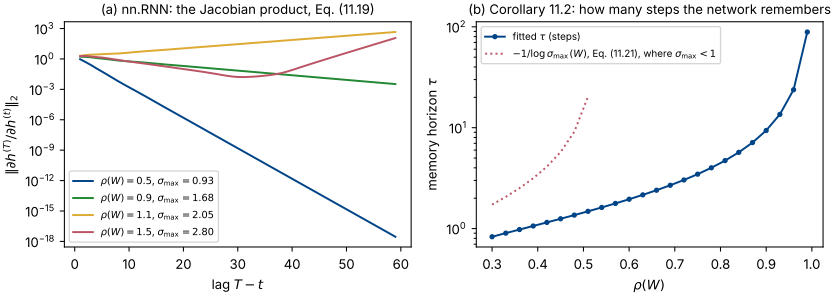
Propagating the gradient back only $k$ steps drops the tail of the product; the error is bounded by

$$\frac{C\, e^{-k/\tau}}{e^{1/\tau} - 1},$$

so $k$ a few multiples of $\tau$ is safe.

But read what this says: truncation is cheap precisely when the recurrence is *not* doing the thing it was introduced for.

# Measured with `torch.autograd` on `nn.RNN` (`week45_gradients.pdf`)




$n_h = 20$, $T = 60$, inputs small enough that $\tanh' \approx 1$, W scaled to a prescribed spectral radius; each curve is `torch.autograd.functional.jacobian` of $h^{(T)}$ with respect to $h^{(t)}$. (a) At $\rho(W) = 0.5$ the influence of the first step on the last has fallen by *eighteen orders of magnitude* over 59 steps. At $\rho = 1.1$ it has grown to 450. (b) The horizon $\tau$ fitted to the slope of (a): below one step at $\rho = 0.3$, ninety at $\rho = 0.99$.

# The numbers behind the figure

```
rho(W) sigma_max mean max(tanh') ||dh_T/dh_1|| bound sigma_max^59 fitted tau
 0.50 0.933 1.0000 2.878e-18 1.705e-02 1.44
 0.90 1.680 1.0000 3.313e-03 1.963e+13 9.38
 1.10 2.053 1.0000 4.547e+02 2.721e+18 -10.63
 1.50 2.800 0.9980 1.152e+02 2.410e+26 -20.40

fitted memory horizon and the lag at which the influence has fallen by 1e-6:
  rho=0.50 tau= 1.44 steps lag(1e-6)= 20
  rho=0.70 tau= 2.79 lag(1e-6)= 39
  rho=0.90 tau= 9.38 lag(1e-6)= 130
  rho=0.99 tau= 88.88 lag(1e-6)=1228
```

The code that produced this is a dozen lines of the notebook: build the layer, scale `weight_hh_l0`, run it, and ask autograd for the Jacobian from each intermediate state to the last.

# Reading the table

- At $\rho(W) = 0.5$ the memory horizon is *one and a half steps*. A network initialised this way has forgotten its input after two steps, and no amount of training on long sequences will change that until the weights themselves grow.
- The bound uses $\sigma_{\max}(W)$, not $\rho(W)$, and the two differ: $\rho = 0.5$ has $\sigma_{\max} = 0.93$, $\rho = 0.9$ has $\sigma_{\max} = 1.7$. Between $\rho < 1 < \sigma_{\max}$ there is a regime of *transient growth with asymptotic decay* – the red curve of (a). That is where carefully initialised recurrent networks actually operate.
- None of this depends on the library. Autograd computes the product of Eq. (11.19) exactly; the vanishing gradient is *a property of the composition*, not of an implementation.

# Two bits per step, and one per cent from the edge

- The logistic function has $\max|\sigma'| = 1/4$ ($\sigma' = \sigma(1-\sigma)$ at $\sigma = 1/2$); tanh has $\max|\tanh'| = 1$ at the origin. A logistic recurrence shrinks the gradient by at least $4^{T-t}$ over a lag $T - t$ whatever W does – two bits per step, $10^{35}$ over sixty steps.
- With tanh, $\tau = -1/\log \sigma_{\max}$: 9.5 steps at $\sigma_{\max} = 0.9$, 99.5 at 0.99, 999.5 at 0.999. A horizon of 100 steps needs $\sigma_{\max} \geq e^{-1/100} \approx 0.990$.
- The growth condition of the theorem is $\mu\sigma_{\max} > 1$, and 0.99 sits one per cent below it. To remember a hundred steps the simple RNN must be tuned to within a per cent of the exploding regime – and training moves W.

That is the design problem the gated cells of next Monday solve. Today: what the simple cell can do when the sequences are short enough, and the one remedy that is free.

# Gradient clipping, Eq. (11.23)

Vanishing and exploding gradients are not symmetric problems. Explosion is the easier of the two, because it is *visible*: the gradient norm spikes and the parameters jump. The standard fix rescales any gradient whose norm exceeds a threshold, keeping the direction:

$$\mathrm{g} \longleftarrow \begin{cases} \mathrm{g}, & \|\mathrm{g}\| \le \theta, \\ \dfrac{\theta}{\|\mathrm{g}\|}\,\mathrm{g}, & \|\mathrm{g}\| > \theta. \end{cases}$$

Proposition 11.3: clipping is a trust region

(i) $\tilde{\mathrm{g}} = \lambda \mathrm{g}$ with $\lambda \in (0, 1]$, so $-\tilde{\mathrm{g}}$ is still a descent direction. (ii) $\tilde{\mathrm{g}}$ is the projection of g onto the ball of radius $\theta$, Eq. (11.24), so a step $\gamma\tilde{\mathrm{g}}$ never moves the parameters further than $\gamma\theta$ – the guarantee of a trust-region method, for free.

# Clipping: what it does not do, and where it lives

### What it does not do

Nothing whatever for *vanishing* gradients: rescaling $10^{-18}$ upwards would amplify rounding noise, not signal. Vanishing gradients need a change of architecture – the gated cells of next Monday.

### In the libraries

`Adam(lr, clipnorm=1.0)` in Keras; `nn.utils.clip_grad_norm_(model.parameters(), 1.0)` between `backward()` and `step()` in PyTorch. Both act on the *whole* gradient vector, all arrays jointly.

### Initialisation

$\sigma_{\max}(W) \approx 1$ puts the network at the edge between the two regimes – the best this architecture can do. Orthogonal or identity initialisations of W are attempts to sit exactly there.

# Four effective ways to train a recurrent network

1. **Long short-term memory.** Make the RNN out of modules designed to remember values for a long time – next Monday.
2. **Hessian-free optimisation.** An optimiser that can detect directions with a tiny gradient but an even smaller curvature (Newton's method of week 37 made affordable).
3. **Echo-state networks.** Initialise the recurrent connections very carefully, so that the hidden state is a reservoir of weakly coupled oscillators – and learn only the read-out V (linear regression, week 35!).
4. **Good initialisation with momentum.** Initialise as in an echo-state network, then learn everything, with momentum and clipping.

In practice: a gated cell, $\sigma_{\max}(\mathrm{W}) \approx 1$ at the start, Adam, and clipping. Before we gate anything, let us see what the plain layer can do when the sequences are short enough.

# The oscillator experiment, 1: the data and the Keras model (Section 11.6.1)

```
def make_windows(x, L=40, stride=5): # x: one trajectory of 800 points
    idx = range(0, len(x) - L - 1, stride)
    X = np.stack([x[i:i + L] for i in idx])[..., None] # (152, 40, 1): inputs x_i..x_{i+39}
    Y = np.stack([x[i + 1:i + L + 1] for i in idx])[..., None] # (152, 40, 1): targets, one step ahead
    return X, Y

model = keras.Sequential([keras.Input(shape=(None, 1)),
                          layers.SimpleRNN(24, return_sequences=True),
                          layers.Dense(1)])
model.compile(optimizer=keras.optimizers.Adam(5e-3, clipnorm=1.0), loss="mse")
model.fit(X_train, Y_train, epochs=300, batch_size=4, verbose=0)
```

- Data: the *single* RK4 trajectory of panel (a), $\eta = 0.2$, $(x_0, v_0) = (1, 0)$, 800 points – the network sees only $x(t)$.
- Task: one-step-ahead forecasting at every step, along 152 overlapping windows of length 40 (stride 5).
- Network: $n_h = 24$, 649 parameters; Adam at $5 \cdot 10^{-3}$, clipping at 1, 300 epochs.

# The oscillator experiment, 2: the same in PyTorch

```
model = Forecaster() # nn.RNN(1, 24) + nn.Linear(24, 1) from Part 4
opt, lossfn = torch.optim.Adam(model.parameters(), lr=5e-3), nn.MSELoss()
gnorms = []
for epoch in range(300):
    for idx in torch.randperm(len(X_train)).split(4):
        opt.zero_grad()
        loss = lossfn(model(X_train[idx]), Y_train[idx])
        loss.backward()
        gnorms.append(float(nn.utils.clip_grad_norm_(model.parameters(), 1.0)))
        opt.step()
```

## The tests

(b) the trajectory from $(0, 1)$ – a different solution of the same equation, out of phase with everything the network saw; (c) an oscillator driven by $F(t) = 0.3\cos(0.7t)$ from $(0.5, -0.3)$, which the network never saw in any form. The split is by whole trajectories: every test trajectory has a new initial condition.

# Two rules for sequential data

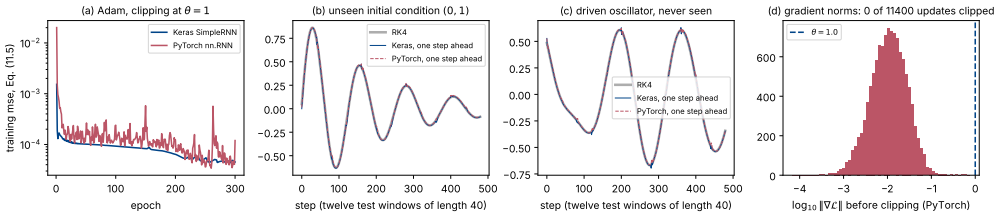
### No shuffling across time

Training and test *windows* may be shuffled, the points inside a window may not – shuffling in time decorrelates the series and leaks the future into the past. `randperm(...).split(4)` and Keras's `shuffle=True` shuffle windows, never steps.

### The cost

Mean squared error of predicted against actual trajectory, Eq. (11.5): a forecasting problem is a regression problem, and we are free to choose among the regression losses of week 35.

# The result (week45_oscillator.pdf)




```
set Keras mse PyTorch mse Var(x) relative (Keras / PyTorch)
train 3.692e-05 1.056e-04 0.0655 0.056% / 0.161%
unseen IC 3.811e-05 1.023e-04 0.0618 0.062% / 0.166%
driven 4.970e-05 1.269e-04 0.1488 0.033% / 0.085%
PyTorch gradient norms before clipping: median 0.011, max 0.77, 0 of 11400 updates clipped
```

# What the result says

- The second row is the interesting one: trained on a trajectory starting from rest at unit displacement, tested on one starting from the origin at unit velocity, the error is *the same as the training error* – six parts in ten thousand of the variance in Keras, sixteen in PyTorch.
- The driven oscillator, never seen, is predicted just as well.
- The network has not memorised a trajectory; it has learned the *update rule*, as Eq. (11.12) said it would – the same payoff parameter sharing gave in week 44: a rule learned once applies everywhere it is valid.
- The two libraries differ by a factor of three in the final error: different random initialisations and different batch orderings. Both are far below anything that matters.
- (d): with $\sigma_{\max}(W) \approx 1$ at the start and sequences of 40 steps, clipping never fired in 11 400 updates. It is insurance.

# Forecasting a sine wave, 1: Keras (Section 11.8.1)

```
time_steps = np.linspace(0, 100, 500); data = np.sin(time_steps); seq_length = 20
X = np.array([data[i:i+seq_length] for i in range(len(data)-seq_length)]).reshape(-1, seq_length, 1)
y = data[seq_length:].reshape(-1, 1) # the next value after each window
split = int(0.8 * len(X)) # split in TIME, not at random

model = keras.Sequential([keras.Input(shape=(seq_length, 1)),
                          layers.SimpleRNN(16), # last state only: (N, 16)
                          layers.Dense(1)])
model.compile(optimizer=keras.optimizers.Adam(1e-2, clipnorm=1.0), loss="mse")
model.fit(X[:split], y[:split], epochs=200, batch_size=32, verbose=0,
          validation_data=(X[split:], y[split:]))
```

Sequence-to-label: 20 values in, *one* value out, so `return_sequences` stays at its default `False` and `Dense(1)` acts on the last state. The last 20% of the wave is the test set.

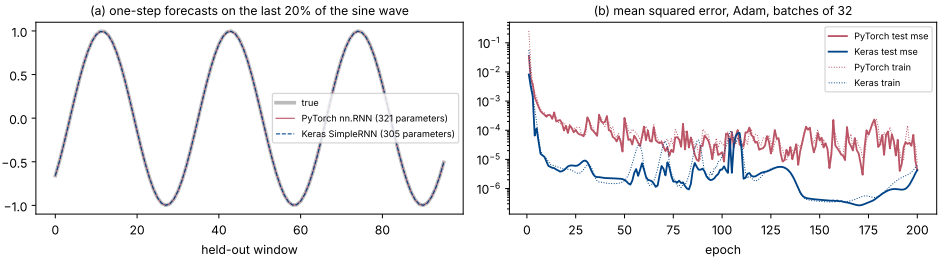
# Forecasting a sine wave, 2: PyTorch

```
class SineRNN(nn.Module):
    def __init__(self, hidden=16):
        super().__init__()
        self.rnn = nn.RNN(input_size=1, hidden_size=hidden, batch_first=True)
        self.fc = nn.Linear(hidden, 1)
    def forward(self, x):
        out, h_T = self.rnn(x) # out: (N, 20, 16), every h_t
        return self.fc(out[:, -1, :]) # read out the last state only

model = SineRNN(); opt = torch.optim.Adam(model.parameters(), lr=1e-2); lossfn = nn.MSELoss()
for epoch in range(200):
    for idx in torch.randperm(split).split(32):
        opt.zero_grad()
        loss = lossfn(model(X[idx]), y[idx])
        loss.backward()
        nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        opt.step()
```

`out[:, -1, :]` is Keras's `return_sequences=False`. The loop is the one of Part 4, unchanged.

# Both reach the same place (`week45_sine.pdf`)




```
PyTorch epoch 200: train mse 0.000008 test mse 0.000004 (321 parameters)
Keras epoch 200: train mse 0.000006 test mse 0.000004 (305 parameters)
rms error on the held-out windows: 0.0021 (both), amplitude 1
```

A test mean squared error of a few $10^{-6}$ is a root-mean-square error of a few $10^{-3}$ on a signal of unit amplitude: both predictions lie on the true curve. The parameter counts differ by sixteen: the second bias vector.

# MNIST read row by row (Section 11.8.2)

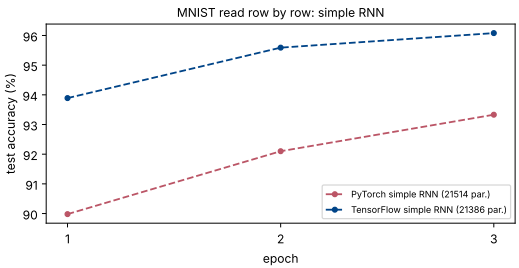
Treat each $28 \times 28$ image as a sequence of 28 rows, each of 28 features, and classify it from the last state – a sequence-to-label task. Not what one does in practice (the recap said why a CNN is the right tool), but a clean demonstration, and a direct comparison with week 44.

```
# Keras
m = keras.Sequential([keras.Input(shape=(28, 28)), layers.SimpleRNN(128), layers.Dense(10)])
m.compile(optimizer=keras.optimizers.Adam(1e-3, clipnorm=1.0),
          loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True), metrics=["accuracy"])
m.fit(x_train, y_train, epochs=3, batch_size=64) # x_train: (60000, 28, 28) in [0, 1]

# PyTorch
class RowNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.rnn = nn.RNN(28, 128, batch_first=True) # 28 steps of 28 features
        self.fc = nn.Linear(128, 10)
    def forward(self, x):
        return self.fc(self.rnn(x)[0][:, -1, :]) # last state -> class scores
criterion = nn.CrossEntropyLoss() # softmax + cross entropy, week 41
```

Compare the CNN program of the recap: the same `Input` – layer – `Dense(10)` pattern, the same loss, the same loop; only the layer in the middle has changed, and with it the way the image is read.

# Row by row: the result (`week45_mnist_rows.pdf`)




```
cell framework parameters test acc.
simple RNN PyTorch 21514 0.9333
simple RNN Keras 21386 0.9608
(CNN, week 44 3241354 0.9906)
```

Three epochs, Adam $10^{-3}$, batches of 64, $n_h = 128$, the full 60 000 images; accuracy on the 10 000 test images.

- A simple recurrent layer with 21 000 parameters reads the image in 28 steps and gets 93–96%: the task is within its memory horizon, and the parameter count is again independent of the image height.
- The CNN of the recap reached 99%: reading an image as a sequence throws away the vertical structure that convolution was built to exploit. The right tool for images is still the CNN; the point here is that the recurrent layer is a drop-in replacement in the code.

# The experiment the theory demands: the adding problem (Section 11.8.5)

Corollary 11.2 says a simple RNN initialised in the usual way has a memory horizon of a few steps. MNIST rows and the oscillator windows were short enough. Testing the claim needs a task where nothing but long-range memory is measured.

### The adding problem

Each sequence has two channels: the first carries values drawn uniformly from $[0, 1]$, the second is zero except for exactly two markers, one in each half. The target is half the sum of the two marked values.

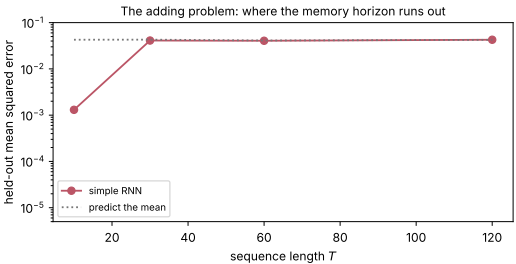
Nothing can be inferred from local statistics – the values are i.i.d. – so a model must hold one number from the first half until the second marker arrives, while ignoring everything in between. Predicting the mean gives a mean squared error of about 0.043; that is the number to beat.

# The adding problem: setup

```
for T in (10, 30, 60, 120):
    core, head = nn.RNN(2, 32, batch_first=True), nn.Linear(32, 1) # two channels in
    params = list(core.parameters()) + list(head.parameters())
    opt = torch.optim.Adam(params, lr=5e-3)
    for epoch in range(40):
        for idx in torch.randperm(3000).split(64):
            opt.zero_grad()
            loss = nn.MSELoss()(head(core(X[idx])[0][:, -1, :]), y[idx])
            loss.backward(); nn.utils.clip_grad_norm_(params, 1.0); opt.step()
```

- 3000 training and 1000 held-out sequences; $n_h = 32$; the same seed, optimiser and epochs for every $T$.
- Sequence-to-label again: the read-out sees `[0][:, -1, :]`, the last state.
- Prediction from Corollary 11.2: a dependency at lag 30 should be invisible to this network.

# Where the horizon runs out (`week45_adding.pdf`)




```
  T baseline simple RNN
 10 0.0427 0.0013
 30 0.0429 0.0410
 60 0.0411 0.0405
120 0.0425 0.0426
```

Held-out mean squared error; the baseline column is predicting the mean.

- At $T = 10$ the simple recurrent network solves the task: 0.0013 against 0.0427.
- At $T = 30$ it has *already failed completely* – an error equal to the baseline is a model that outputs the mean and nothing else – and it never recovers at 60 or 120. The gap between $T = 10$ and $T = 30$ is where the horizon of Corollary 11.2 runs out.
- Next Monday we change the cell, keep everything else in this code, and run the same table again.

# Where this is going: tomorrow, Project 2, next week

### Tomorrow: Project 2

The Tuesday session is yours for Project 2 (deadline **Monday November 9**): open questions first, then work in your groups with us circulating. Bring your results and the parts of the report you are unsure about. No weekly exercises this week.

### If you want to use today in Project 2

You do not have to. A group that treats the regression part as a one-step forecasting problem – the Runge function sampled along $x$ as a sequence – and compares a Keras or PyTorch recurrent network against its dense network is welcome to, cross-validated the same way and compared honestly.

### Reading

Chapter 11, Sections 11.1–11.6 and 11.8 (11.7 is next week); Goodfellow et al. Chapter 10; Raschka et al. Chapter 15. The code of today is in `week45.ipynb`.

### Next Monday, week 46

First hour: the gated cells – long short-term memory and the gated recurrent unit, Section 11.7 – measured the same way, run on the adding problem, and on sound (spoken digits). Then decision trees and ensemble methods (Chapter 7).

# Summary of week 45

## From convolution to recurrence

A CNN shares its parameters along the spatial axes; a recurrent network shares them along the time axis. The simple RNN is a learned dynamical system – the oscillator showed it is a learned time-stepping scheme – and in the libraries it is one layer, `SimpleRNN` or `nn.RNN`, fed a 3-tensor `(samples, timesteps, features)`, in the same `Sequential` / training-loop pattern as the CNN.

## How it is trained, and why that is hard

`loss.backward()` and `model.fit` perform backpropagation through time exactly: a linear backward pass whose gradient across a lag is a product of Jacobians – $10^{-18}$ over 59 steps at $\rho(\mathrm{W}) = 0.5$, a memory horizon of 1.4 steps, measured with autograd. Clipping handles explosion for free and does nothing for vanishing.

## What the simple cell can and cannot do

Trained on one damped oscillation it predicted an unseen initial condition and an unseen forcing at the training error: it learned the rule, not the trajectory. It reads MNIST in 28 steps at 93–96%. And on the adding problem it fails from $T = 30$ – exactly where Corollary 11.2 says its memory runs out. Next Monday: the cell that does not.